Bachelor Thesis

Nsak as Framework for Scenario Based Network Security

Course of study	Bachelor of Science in Computer Science
Author	Frank Gauss (gausf1) and Lukas von Allmen (vonal3)
Advisor	Wenger Hansjürg

Version 1.0 of March 26, 2026

Technik und Informatik

Abstract

One-paragraph summary of the entire study – typically no more than 250 words in length (and in many cases it is well shorter than that), the Abstract provides an overview of the study.

Contents

1 Introduction

Computer networks are constantly exposed to evolving cyber threats ...

The *Network Swiss Army Knife (NSAK)* is a framework designed to execute orchestrated security scenarios consisting of reusable attack drills in a controlled network environment. The framework aims to support automated security testing and adversary emulation on network edge devices.

The development of the NSAK framework originates from previous project work, in which the core architecture and the basic scenario execution model were developed. This thesis builds upon this foundation and extends the framework by investigating how AI agents can support automated red and blue team scenarios.

...

1.1 Motivation

Hypothesis: AI can enhance red and blue team simulations in a network environment by generating and executing scenarios within the NSAK framework and assisting in the evaluation of detection results.

1.2 Predecessor Project: "Project 2"

Describe what we did in project 2 and what project 2 is (BFH module, possible predecessor to bachelor thesis)

1.3 Project Goals

For a goal to be considered **S.M.A.R.T.**, it must fulfill the following criteria:

- **Specific**: What do we want to achieve?
- **Measurable**: What are the success criteria?
- **Achievable**: How do we plan to achieve this goal?
- **Relevant**: Why is it important to achieve this goal?

- **Time-bound**: Goals categorized as *must* have to be achieved within the thesis, while *should* or *could* goals are considered optional or nice to have.

1.3.1 Research Goals

Compare Frameworks (must)

To identify the USP (unique selling point) of the NSAK framework, we will analyze and compare existing security emulation frameworks. First, relevant frameworks such as MITRE Caldera and Atomic Red Team will be identified and reviewed. Based on this analysis, the identified concepts will be compared with the architecture of the NSAK framework. The comparison will focus on how scenarios and drills are defined, how configuration and parameter handling are structured, how reusable components are organized, how automation is achieved, and which AI capabilities they provide.

The analysis and comparison will be based on the following properties:

- Concepts
- Modularity
- Configurability
- Automation
- AI Capabilities

Success criteria:

- At least two security emulation frameworks are identified and reviewed
- A matrix comparing the specified properties of the selected frameworks is created
- The USP of the NSAK framework is identified

AI Integration (must)

The goal of this research is to investigate how AI (artificial intelligence) can support automated security testing and adversary emulation. The research evaluates potential integration points where AI could enhance the scenario execution, automation, decision-making, and analysis capabilities of the NSAK framework.

Success criteria:

- How is AI currently used in cybersecurity automation?
- Which tasks in adversary emulation can be supported or automated by AI?
- How could AI interact with the NSAK scenarios and drills (e.g., Agents, MCP)?
- What architectural changes would be required to integrate AI into NSAK?

1.3.2 Framework

Configuration Management (must)

The goal is to extend the NSAK framework, so that scenarios and drills can expose configurable parameters to increase the flexibility of scenarios and reusability of drills. Parameters can be provided via command-line arguments or YAML files and are resolved by the framework into a unified run configuration used for scenario execution.

Success criteria:

- Scenarios and drills can expose configurable parameters
- The available parameters can be listed by the CLI with the `--help` option
- Parameters can be provided via CLI arguments or YAML files
- The framework resolves the inputs into a run configuration (internal data structure).

REST API (TBD)

Adding a REST API to the NSAK framework was already proposed in the predecessor project, as it greatly enhances interoperability with other tools. The priority of this goal depends heavily on the outcome of the goal 1.3.1 and on whether we want to fulfill the goal 1.3.2, which is prioritized as “could”. If the evaluated AI integration requires a REST API (e.g., MCP), this goal needs to be prioritized as “must”; it will be prioritized as “could”.

Success criteria:

- A REST API specification with feature parity to the CLI is created.

- The REST API is implemented according to the specification.
- The API is documented (Open API) and usable by external clients.
- The REST API remains optional and does not replace the CLI-based interaction.
- Authentication and Authorization are explicitly out of scope.

MCP Server (TBD)

The MCP Protocol allows the AI to interact with applications in a standardized way - and could therefore be necessary for implementing AI-based scenarios. If the outcome of the research task 1.3.1 highlights the necessity to specify/expose an MCP-Server to interact seamlessly with the NSAK framework, the implementation is a (must), otherwise a (could).

- A MCP-Server is specified and implemented
- Documentation how to connect to the MCP is provided.
- A form of authentication is provided.

Web GUI (could)

The goal is to design and implement a web-based graphical user interface that allows users to interact with the NSAK framework in a user-friendly way. The Web GUI will enable management of scenarios and drills, triggering scenario execution, and visualization of execution results and system status.

Success criteria:

- A simple Web GUI that interacts with the REST API has been implemented.
- The Web GUI displays a list of available scenarios and their states.
- The scenario's results can be viewed on the Web GUI
- Scenarios can be configured and executed via the Web GUI
- Authentication and authorization are explicitly out of scope.

1.3.3 Scenarios

AI – Purple Team (must)

The goal is to implement artificial intelligence according to the result of 1.3.1 and to explore the use of AI to support red, blue, and purple team exercises in the NSAK framework. The AI will be evaluated for its potential to assist in executing attacks in a scenario and analyzing detection results. The findings will help assess how AI could enhance and coordinate red and blue team activities across a network via the NSAK device.

Success criteria:

- The AI is integrated into NSAK as a scenario or as part of the framework, depending on the result of 1.3.1.
- The AI can execute CLI or Python tools to directly or indirectly interact with the attached network.
- An operator can interact with or stop the running AI.
- The result artifacts are stored and can be presented or reused for further scenarios.

Reproducible Test Environment (must)

The goal is to design and implement a reproducible and safe test environment for the NSAK framework that enables consistent execution and evaluation of scenarios and drills. The environment should provide a stable basis for testing and comparison of manual and AI-assisted scenarios.

Success criteria:

- The test environment can be set up reproducibly using a defined configuration or setup procedure with Infrastructure as code (e.g., Ansible)
- NSAK scenarios and drills can be executed consistently within the environment.
- The environment supports both manual and AI-assisted scenarios.
- Scenario results can be collected and analyzed for evaluation purposes.

Network Discovery – Red Team (must)

The goal is to implement a reference red team scenario for network discovery within the NSAK framework. The scenario should simulate reconnaissance activities to identify hosts, services, and network characteristics, and the results should be comparable to the AI-Agent approach.

Success criteria:

- A red team network discovery scenario is implemented in the NSAK framework.
- Discovered network information is collected as artifacts.
- The scenario can be executed reproducibly within the test environment.
- The results can be compared with those generated by the AI-assisted approach.

Intrusion Detection – Blue Team (should)

The goal is to define and implement a blue team reference scenario for intrusion detection within the NSAK framework. The scenario should allow observation and evaluation of suspicious network activity to assess detection capabilities in the test environment and compare the results with those of the AI approach.

Success criteria:

- A blue-team intrusion-detection scenario is implemented in the NSAK framework.
- Detection events or alerts can be observed and documented during scenario execution.
- Detection results can be compared with results from AI-assisted scenario execution.

1.3.4 Evaluation Goals

AI Scenario Evaluation (must)

The objective is to determine the effectiveness of the AI-based scenario introduced by the goal 1.3.3 within the network environment defined by the goal 1.3.3 The evaluation will assess whether the AI scenario successfully executed the expected red and blue team activities for the provided prompts and the quality of the results.

Success criteria:

- A methodical approach for the assessment is evaluated and documented.
- The results of the AI scenario for red and blue team activity are assessed.
- There is a conclusion on the effectiveness of the AI scenario for red and blue team activities.

AI vs Engineered Scenarios (must/should)

The goal of this evaluation is to compare the quality of AI-based red (network reconnaissance) (must) and blue (network intrusion detection) (should) team scenarios with that of manually engineered scenarios.

The comparison focuses on criteria such as execution reliability, reproducibility, and coverage. The results will help determine the potential benefits and limitations of AI-supported scenarios and provide insights into how AI could enhance future versions of the NSAK framework.

Success criteria:

- A methodical approach for the comparison is evaluated and documented.
- The results of the AI scenario are compared against the respective results of the red team scenario (must)
- The results of the AI scenario are compared against the respective results of the blue team scenario (should)
- There is a conclusion on the effectiveness of the AI scenario compared to manually engineered scenarios.

1.3.5 Out of Scope / Optional Feature

At the project management meeting on 05.03.26 (see Table **??** in Appendix A.7), the initial project scope was discussed. During planning, the team proposed using AI-based scenarios and comparing them with the implemented versions, which required reassessing certain milestones.

Running AI-based scenarios does not require a REST API or GUI. A reproducible test setup and further research are needed to properly evaluate this approach.

2 Related Research

2.1 Framework Comparison

Previous research, such as 'SOK: A Survey of Open-Source Threat Emulators', compares multiple types of threat emulation frameworks using both quantitative and qualitative metrics [1]. Among the evaluated frameworks, Atomic Red Team, Mitre Caldera, and Metasploit received high scores in categories as environments and operator, environment configuration and scenario execution.

n the following section, the selected frameworks are briefly introduced, and the unique selling point of the Network Swiss Army Knife (NSAK) is highlighted. As a point of reference, the cybersecurity community increasingly relies on the concepts of MITRE ATT&CK, which has established itself as a standard framework in the field since its introduction in 2014 [2, 3].

The MITRE ATT&CK is a globally accessible knowledge base of adversary tactics and techniques based on real-world observations [2]. The knowledge base is structured into TTP (Tactics, Techniques, and Procedures), which are organized into 14 attack behavior groups. The TTP is getting continuously updated and extended, which helps to defend against the capabilities of threat actors.

The evaluation of the proposed frameworks is based on information obtained from official documentation [4–6] and the findings of Zilberman et al. [1].

Characteristic	Metasploit Framework	Atomic Red Team	MITRE Caldera
Primary Purpose	Exploitation and penetration testing framework	Security testing through small atomic ATT&CK tests	Automated adversary emulation platform
Relation to MITRE ATT&CK	Partial mapping via modules	Direct mapping to ATT&CK techniques	Built around ATT&CK adversary emulation
Automation Level	Medium (scripts and modules)	Low (manual execution of tests)	High (automated attack chains)
Main Functionality	Exploit development, payload delivery, post-exploitation	Execute small ATT&CK technique tests	Simulate adversary campaigns across hosts
Architecture	Modular framework with exploits, payloads, and auxiliary modules	Collection of scripts/tests organized by ATT&CK techniques	Client-server platform
Typical Use Case	Penetration testing and vulnerability validation	Detection validation and security control testing	Red team automation and purple team exercises
Difficulty Level	Medium–High	Low–Medium	Medium–High

Table 2.1: Comparison of Metasploit, Atomic Red Team, and MITRE Caldera ([1, 4–6])

Table formatting

As shown in Table 2.1, the key characteristics of the frameworks are differ greatly. **Atomic Red Team** provides libraries of attack scripts that implement individual Tactics, Techniques, and Procedures of MITRE ATT&CK. Each attack can be executed independently or chained together into a larger attack scenario. **Mitre Caldera** is a complex, agent based adversary simulation tool, whereas **Metasploit** is a framework that provides exploits, payloads and auxiliary modules. Further details of the frameworks are presented in the following sections. 2.1.12.1.22.1.3

2.1.1 Atomic Red Team

The Atomic Red Team repository provides documentation and configuration files for each technique. For every TTP, a YAML Ain't Markup Language (YAML) configuration file defines the execution parameters, while an enhanced Markdown file contains a human-readable description of the attack. This documentation includes a short description of the technique, execution instructions, and additional contextual information [5].

The framework can be executed directly from the command line and provides features such as logging, detailed error messages, and automatic prerequisite checks. If a technique cannot be executed, for example due to operating system constraints, the framework reports the issue and can optionally attempt to install the required prerequisites [5].

Zilberman et al. recommends deeper cybersecurity knowledge to use Atomic Red Team framework effectively [1]. The Open Source mentality, expandability and easily usable scripts can be a great asset for an independent framework such as the NSAK.

2.1.2 Caldera

Mitre Caldera is also built on the MITRE ATT&CK knowledge base and functions as a command-and-control center. A central server controls agents running on the target system, executes attacks called abilities, and sends the operation results back to the server. Every Ability is a single attack step built on the Tactics, Techniques, and Procedures. The framework ships with a graphical user interface and is primarily used for red team attack simulation, purple teaming, and detection engineering. The additional features, like a training plugin for educational purposes, include facts that can be configured as dynamic safe values, which can be stored and parsed in later steps [4].

With its provided interface and functionality, Caldera is most suitable as a threat emulator, offering a wide range of built-in features, including lateral movement and Command and Control (CNC) communication [1].

2.1.3 Metasploit

Metasploit is a widely used penetration testing framework for testing or attacking systems and exploiting known vulnerabilities. Metasploit follows a sequence of steps to identify a vulnerability in a target system, select a vulnerability for the attack, select an exploit that uses this vulnerability, and deliver a payload that runs on the target system. The configuration via the Command Line Interface (CLI) requires knowledge of the target network and system and requires partial configuration [6, 7].

According to Zilberman et al., an operator requires extensive cybersecurity knowledge to effectively use the framework. To support the operator, the GUI “Armitage”, which is not further evaluated in this work, can be used [1].

2.1.4 NSAK

The NSAK framework, as an independent universal Network Swiss Army Knife, is not confined and can leverage, if needed, different sets of attack frameworks. An approach could be to use an Artificial Intelligence (AI)-Enhanced Network Discovery Scenario, which provides the necessary information to configure a Metasploit attack that leads to an attack operation in Caldera and an atomic red team detection script. Hardware-based isolation and network position provide flexibility and should help reduce the operational costs of red and blue teams by reducing simple configuration overhead [8].

2.2 AI-Research

2.2.1 AI and Language Models

Language Model (LM) describes the technique for advancing language intelligence in machines and its development history can be divided into several stages. The first stage, Statistical Language Model (SLM) emerged around the 1990s, and the basic idea is to assume, that such a model can predict the next word. But the fact that an exponential number of transition probabilities are needed, caused a lack of accuracy and led limited success.

To overcome these limitations, Neural Language Model (NLM) was introduced to learn a distributed representation of words and capture more complex patterns, leading to a major breakthrough. Building on this, the advancement of pre-trained language models Pre-trained Language model (PLM), which are first trained on large datasets and then fine-tuned for specific tasks, represented another significant development.

Building on this development, Large Language Model (LLM)s further increases the scale of PLMs by expanding parameter count and computational power [9].

In large language models, parameters are the trainable numerical values that define how the model processes input and generates output. During training, these parameters are continually adjusted to enable the model to learn patterns and language context [10]. The models most of us use today, such as ChatGPT, Claude and Gemini are pre-trained LLMs used for language inference.

These advances led to Model Context Protocol (MCP), a protocol that allows tools and services to expose an interface specifically for AI. Further, the introduction of AI-agents is enabling a new level of autonomy for achieving concrete goals with LLMs. The combination of these technologies is now resulting in systems, where LLMs empower autonomous agents capable of decision-making to execute complex tasks.

2.2.2 MCP

Anthropic, a US software company focused on AI research, introduced the protocol in 2024. The MCP was introduced as an open source standard that enables AI assistants to connect to external systems, including databases, business tools, and development environments [11]. Dynamic tool discovery and schema negotiation is helping clients and agents to access these functions in a standardized manner and allows autonomous interaction. Additionally, MCP allows information to flow in both directions, so the model can send requests, and tools can send updates or events.

This architectural approach shifts the AI integration from static toward an interoperable ecosystem of AI-enabled services. On the other hand, the flexible design and open source approach of MCP unlocks multiple novel and systematic security risks that are in need of enhanced security and governance [12].

2.2.3 Agentic-AI

Agentic Artificial Intelligence (Agentic-AI) can be understood as an advanced machine intelligence that is capable of perceiving, reasoning, and acting with a certain level of autonomy. Traditional systems and agents are more limited to executing predefined commands, whereas Agentic-AI is not restricted in this way. Due to their ability to adapt in real time, they can continuously refine their knowledge, techniques, and data. The ability to access multiple inputs, from natural language processing to sensor data, allows Artificial Intelligence Agent (AI Agent)s to develop a good understanding of their environment [13].

Multi Agent System (MAS) defines a way in which multiple specialist agent coordinate and cooperate with each other to solve a task, that would not be solvable to one agent alone.

Therefore, the agents need a form of interaction that allows them to act reactively, often through a shared mental model, which helps create a common understanding of the complex tasks to be solved. To implement a MAS efficiently, modern design principles such as encapsulation and modularity are important, which are helping to balance resource usage of the system and LLM [14]

Huang et al. provide evidence that Agentic-AI and MAS are capable of solving complex tasks across various sectors, including security and banking. However, these systems also introduce new potential vulnerabilities, both within AI Agent systems themselves and through their interactions with external environments.

The identified vulnerabilities can be categorized into two types: accidental and deliberate. Accidental vulnerabilities include software bugs, logical errors introduced by agents, hardware malfunctions, and poor data quality. Deliberate

attacks, on the other hand, encompass adversarial attacks targeting LLMs, data poisoning, and model theft.

To mitigate these risks, a comprehensive framework consisting of governance mechanisms, proactive monitoring, regulatory compliance, and rigorous testing is required to ensure the secure and reliable operation of agentic systems [15].

Fazit MCP and Agentic AI

In the cybersecurity podcast Risky Business, it is mentioned that MCP is "dead". The open-source strategy of Anthropic (2.2.2) has led to the development of multiple MCP implementations that were not designed with a strong focus on security, but rather on rapid adoption of a novel technique.

This provocative claim is based on the observation that, in order to handle the excessive MCP tooling, AI Agents may instead rely on direct access to a root shell, as it provides a more efficient way of interaction [16].

How and why is this relevant for our thesis? how could we implement it? To which findings should we pay attention? What can we ignore for now?

Evaluated Frameworks and MCP

Caldera uses a MCP plugin as a bridge between the model and the Representational State Transfer Application Programming Interface (REST-API), allowing the agent to plan and issue commands in a manner similar to a human operator.

Caldera introduces three main research areas. Firstly, a planner determines the order and selection of abilities and can dynamically construct new adversaries based on user requirements through prompt-based interaction.

Secondly, an adversary factory model is used to generate adversaries tailored to the operator's specific use cases.

Thirdly, a forward-deployed agent is enhanced with Retrieval Augmented Generation (RAG) for Cyber Threat Intelligence (CTI), leveraging Structured Threat Information Expression (STIX)-based data to create reproducible and testable adversaries [17].

What about the other frameworks? What about AI Agents? What can we learn from their approach? How does it affect our strategy?

3 Concepts

4 Methodology

5 Implementation

5.1 Configuration Management Implementation

5.1.1 Device Configuration Management

Create New Device Resource Type

In the context of "Project 2"1.2 this resource type was already defined conceptionally and the BananaPI R4 was added as an example in the library under `lib/devices/bananapi_r4`. But the resource type was not actually added to the NSAK core and be able to manage device configurations we need to implement the Device resource type first. Analog to the other resource types, a yaml specification for defining devices in the library and multiple classes for representing and managing devices in the core are required. The listing 1 shows the initial specification for a device resource, which will get extended later on with a section for configuration management.

```yaml
device: <str:resource-version>

metadata:
  id: <str:device-id>
  name: <str:device-name>
  description: <str:device-description>
  author: <str:author-email>
  repository: <str:resource-repository>
```

Listing 1: Device Resource YAML Specification

In the following table 5.1 the required Python classes are listed and described. The classes will be located under `src/nsak/core/device` within their respective files.

In "Project 2"1.2 it became already clear, that the described classes share a lot of boilerplate code between each other. For this reason the base classes described in 5.1 where added to get rid of the duplicated logic, which leads to better compliance with the Don't Repeat Yourself principles. The classes will be located under `src/nsak/core/resource` within their respective files.

To complete the integration of the device resource, the new functionality must be exposed via CLI. The available bash commands are shown in the following listing 2.

Class	File Name	Description
Device	device.py	A dataclass, which represents a device resource as objects during runtime (usually quite similar to the yaml specification).
DeviceLoader	device_loader.py	This class returns Device objects and is used by the DeviceManager to find and load device resources from the library.
DeviceManager	device_manager.py	The manager class implements the business logic and exposes an API, which can be used by the CLI.

Table 5.1: Device Resource Classes

Class	File Name	Description
Resource	resource.py	Base class for all resources: Scenario, Drill, Environment and Device.
ResourceLoader	resource_loader.py	Base class which handles searching and loading resources from the filesystem.
ResourceManager	resource_manager.py	Base class which defines common methods for all resource manager classes.

Table 5.2: Resource Python Classes

```
# Show usage, options and subcommands for the device resource
nsak device --list

# List all devices found in the resource library
nsak device list

# Output:
❯ nsak device list
ID           NAME         PATH
bananapi_r4  BananaPI R4  <repository>/lib/devices/bananapi_r4
```

Listing 2: NSAK CLI: List Devices

Device Configuration Management Implementation

To allow a device to expose its configuration options, the YAML specification needs to be extended accordingly. For now, the focus is only on network configuration as this is the most important information for the scenarios and drills, but the specification was done in such a way that other sections can be added later on. The following excerpt 3 shows the specification for the configuration section of the device resource YAML.

```yaml
device: <str:resource-version>

metadata:
  ...

configuration:
  network:
    ethernets:
      <str:intreface-name>:
        addresses:
          <str:ip-address-with-subnet>:
            is_target: <boolean>
            is_management: <boolean>
```

Listing 3: Device Resource Configuration YAML Specification

The following code snippet located under `src/nsak/core/device/device_loader.py` 4 shows how the YAML configuration gets parsed to the respective datastructures defined in `src/nsak/core/device/device.py` 5.

```
class DeviceLoader(ResourceLoader[Device]):
    """
    Finds and loads devices from the library paths.
    """

    ResourceClass = Device

    @classmethod
    def _parse_device_configuration(
        cls, data: dict[str, Any]
    ) -> DeviceConfigration | None:
        if "configuration" not in data:
            return None

        ethernets = data["configuration"].get("network",
        ↪  {}).get("ethernets", {})

        return DeviceConfigration(
            network=NetworkConfiguration(
                ethernets={
                    interface: EthernetConfiguration(
                        name=interface,
                        addresses={
                            ip: IPConfiguration(
                                ip=ip_interface(ip),

                                ↪  is_target=address.get("is_target",
                                ↪  False),

                                ↪  is_management=address.get("is_management",
                                ↪  False),
                            )
                            for ip, address in
                            ↪  ethernet.get("addresses",
                            ↪  {}).items()
                        },
                    )
                    for interface, ethernet in ethernets.items()
                }
            )
        )

    @classmethod
    def _to_resource(cls, data: dict[str, Any], path: Path) ->
    ↪  Device:                  21
        """
        Creates a Device object from a dict containing the
        ↪  device's metadata.
        """
        return cls.ResourceClass(
            id=str(data["metadata"]["id"]),
            name=str(data["metadata"]["name"]),
            path=path,
```

```python
@dataclasses.dataclass(frozen=True, kw_only=True)
class IPConfiguration:
    """
    Represents an ip configuration on an interface.
    """

    ip: IPInterface
    is_target: bool
    is_management: bool


@dataclasses.dataclass(frozen=True, kw_only=True)
class EthernetConfiguration:
    """
    Represents an ethernet node.
    """

    name: str
    addresses: dict[str, IPConfiguration]


@dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
class NetworkConfiguration:
    """
    Represents the networking part of the configuration, heavily
    ↪  inspired by netplan.
    """

    ethernets: dict[str, EthernetConfiguration]


@dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
class DeviceConfigration:
    """
    Represents the device configuration.
    """

    network: NetworkConfiguration


@dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
class Device(Resource):
    """
    Represents a device.
    """

    id: str
    name: str
    path: Path
    author: str
    repository: str
    configuration: DeviceConfigration | None = None
```


Listing 5: Device Configration Datastructures

In the following listing shows the newly available CLI commands for managing device configurations 6:

```
# List all subcommands for the Device resource
nsak device --help

# List all available devices
nsak device list

# Show the device details and configuration
nsak device show <str:device-id>

# Load a device and its configuration
nsak device load <str:device-id>

# Show the currently loaded device
nsak device loaded

# Reset the loaded device
nsak device unload
```

Listing 6: Device Configuration CLI Management

5.1.2 Drill Configuration Management

5.1.3 Scenario Configuration Management

5.1.4 Configuration via CLI

5.1.5 Configuration via CLI –config_file

5.1.6 Configuration via interactive CLI Dialog

6 Evaluation

7 Discussion

8 Conclusion

Bibliography

[1] Polina Zilberman, Rami Puzis, Sunders Bruskin, Shai Shwarz, and Yuval Elovici. Sok: A survey of open-source threat emulators. arXiv preprint, 2020.

[2] Blake E. Strom, Andy Applebaum, Doug Miller, Adam Nickels, Cody Pennington, and Chris Thomas. Mitre att&ck: Design and philosophy. Technical report, MITRE Corporation, 2020.

[3] Bader Al-Sada, Alireza Sadighian, and Gabriele Oligeri. Mitre att&ck: State of the art and way forward. *ACM Computing Surveys*, 2023.

[4] MITRE Corporation. Mitre caldera documentation, 2024.

[5] Red Canary. Atomic red team, 2024.

[6] Rapid7. Metasploit framework documentation, 2024.

[7] David Kennedy, Jim O'Gorman, Devon Kearns, and Mati Aharoni. *Metasploit: The Penetration Tester's Guide*. No Starch Press, 2011.

[8] Frank Gauss and Lukas von Allmen. Nsak (network swiss army knife). https://github.com/nsak-team/nsak, 2026. Accessed: 2026-01-10.

[9] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2):1–124, 2023.

[10] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.

[11] Anthropic. Model context protocol. https://www.anthropic.com/news/model-context-protocol, 2024. Accessed: 2026-03-18.

[12] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers, 2025.

[13] K Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[14] J Huang K Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[15] K Jackson C Hughes J Huang, K Huang. Ai agent safety and security considerations. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[16] Risky Business Feature Podcast. ...mcp is dead. https://risky.biz/, 2026. Podcast episode , Accessed: 2026-03-18.

[17] Ethan Michalak and Mark Perry. Connecting llms to caldera with the mcp plugin. https://medium.com/@mitrecaldera/connecting-llms-to-caldera-with-the-mcp-plugin-da1450254827, 2025. MITRE Caldera Blog, Accessed: 2026-03-16.

[18] Advisera. Iso 27001 risk assessment, treatment, and management: The complete guide, 2025. Accessed: 2026-03-04.

[19] Simplr. General recommended vram guidelines for llms, 2025. Online; accessed 10 March 2026.

List of Figures

List of Tables

List of Listings

Glossary

This document is incomplete. The external file associated with the glossary 'main' (which should be called `thesis.gls`) hasn't been created.

Check the contents of the file `thesis.glo`. If it's empty, that means you haven't indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can't be generated. If the file isn't empty, the document build process hasn't been completed.

If you don't want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

  ```
  \usepackage[automake]{glossaries-extra}
  ```

- Run the external (Lua) application:

  ```
  makeglossaries-lite.lua "thesis"
  ```

- Run the external (Perl) application:

  ```
  makeglossaries "thesis"
  ```

Then rerun LaTeX on this document.

This message will be removed once the problem has been fixed.

A Project Management

A.1 Stakeholders

Hj. Wenger: Instructor and Network Specialist
Urs Keller: Expert and Entrepreneur

A.2 Agile Management Process

The project followed an interactive development approach using Gitlab for issue tracking and planning. We defined our project management in a Scrum-like manner, but we individualized the process according to the projects scope and our needs. Work items where organized into epics, milestones and issues. Each iteration lasts two weeks. We decided to hold the following Scrum ceremonies after each sprint:

Sprint Planning	**Sprint Review**	**Sprint Retro**	**Checkpoint**
Select milestone with sprint goal	Close open tickets	Reflect on sprint	Present sprint results
Check issues for DoR and obstacles	Gather team feedback	Identify improvements	Get stakeholder feedback
Ensure issues are clear	Validate delivered features	Discuss what went well	Define next tasks
Create sprint backlog	Review progress	Discuss issues/problems	Discuss issues/problems

Table A.1: Key Scrum Meetings

The workflow used for ticket tracking consisted of the following stages:

A.2.1 Issue Lifecycle:

Buckets	Description
Backlog	List of all tasks and features planned for the project.
Todo	Tasks selected for the current sprint that are ready to be worked on.
Q&A	Tasks waiting for clarification, review, or testing.
Done	Completed tasks that meet the defined acceptance criteria.

Table A.2: Issue Board

A.2.2 Definition of Ready (DoR)

Category	Criteria
Clarity	▶ Clear and precise title ▶ Objective or problem described in two to three sentences ▶ Scope clearly defined
Scope	▶ Affected module specified (drill, scenario, core, container, frontend) ▶ External dependencies identified
Validation	▶ Testable acceptance criteria defined ▶ Expected behavior clearly specified

Table A.3: Definition of Ready Criteria for Tickets

A.2.3 Definition of Done (DoD)

Category	Criteria
Implementation	▶ All acceptance criteria fulfilled ▶ Container execution verified ▶ Pre-commit hooks pass ▶ Logging and error handling consistent
Testing	▶ Feature or bug fix tested ▶ Acceptance criteria verifiable (automated or manual) ▶ Relevant edge cases considered ▶ Execution reproducible
Documentation	▶ README updated if required ▶ Architecture changes documented ▶ Thesis relevance briefly noted ▶ Limitations documented ▶ Required documentation stored for thesis or appendix

Table A.4: Definition of Done Criteria for Tickets

A.3 Project Structure

This section describes the projects structure and list the goals as milestones linked to their respective page in GitLab.

A.3.1 Milestones

A milestone represent crucial steps toward the overall project goal. Per our definition a milestone has at least one epic and the mandatory milestones are listed in TableA.5

GitLab Milestones
Milestone Project Management
Research AI Integration
Research: Compare Frameworks
NSAK Framework: Configuration Management
NSAK Scenario: Network Discovery - Red Team
NSAK Scenario: AI - Purple Team
Reproducible Test Environment

Table A.5: Project Milestones in GitLab

A.3.2 Epics

consist of multiple stories and contribute to fulfilling milestones.

A.3.3 Issues

Each Story is assigned a weight, sprint, milestone and epic and are usually represented by an “Issue” in gitlab.

A.4 Sprints / Iterations

A.4.1 Planning

“Task” and “Issue”, the terms are often used interchangeably with “Stories”. Stories must meet the A.2.2 before implementation and A.2.3 before closing.

A.5 Planning and Estimation

The following estimation strategy was used:

- 1 hour corresponds roughly to 1 weight point
- Iteration length: 2 weeks
- Weights per iteration: 40h per person = 80 weight per iteration
- Sprint planning duration: 1.5 hours per iteration
- Sprint review duration: 1 h per iteration

- Sprint retro duration: 1 h per iteration

A.6 Project Roadmap

To keep track of the overall progress in the project, the project team uses the "Roadmap" feature in GitLab. After each sprint planning meeting we create a new screenshot of the roadmap and add it to the according sprint section in A.7. The first sprint planning is an exception, because at this point the diagram would have been empty.

A.7 Sprint Ceremonies

A.7.1 Sprint 1: 19.02 –04.03.2026

Sprint Planning 1: 19.02.2026

For the planning of sprint 1 we derived all issues from the milestone project management A.5 and had an overall weight from 89.

Sprint Review 1: 10.03.2026

All deliverables were fulfilled. However, the deadlines of the milestone had to be adjusted because project management required more time than initially expected. The milestone was successfully reached, and the board was cleared in preparation for the next sprint. All issues were reviewed and moved to the *Done* column.

Figure A.1: Checklist and Burndown Sprint1

Sprint Retro 1: 10.03.2026

Key Takeaways:

- Maintain clear and consistent communication with the instructor.
- Be aware of maintaining a healthy work–life balance during the project.
- Apply lean project management practices to keep the process efficient and focused.

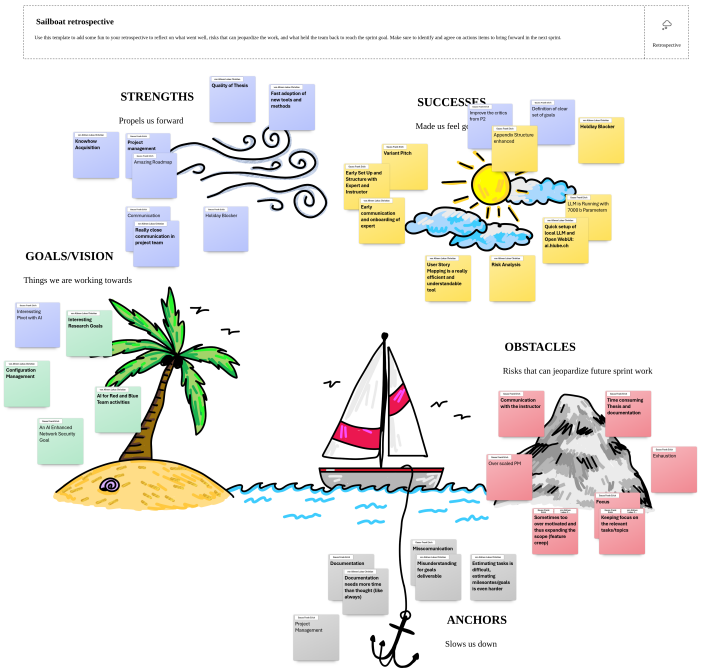

Figure A.2: Sprint Retro 1

A.7.2 Sprint 2: 05.03 –18.03.2026

Sprint Planning 2: 05.03.2026

The current state of the project roadmap is shown in the following screenshot A.3.

Week	Responsible	Task
Week 1	Lukas	Configuration management implementation
Week 1	Frank	Research and comparison of frameworks
Week 1	Team	AI workshop on Thursday and writing of the related issues
Week 2	Team	AI research and evaluation
Week 2	Team	Preparation and pitch of the project deliverables

Table A.6: Sprint Planning 2

Sprint Review 2: 19.03.2026

Add Sprint Review Description

Add Milestone Review/Conclusion: Framework Comparsion

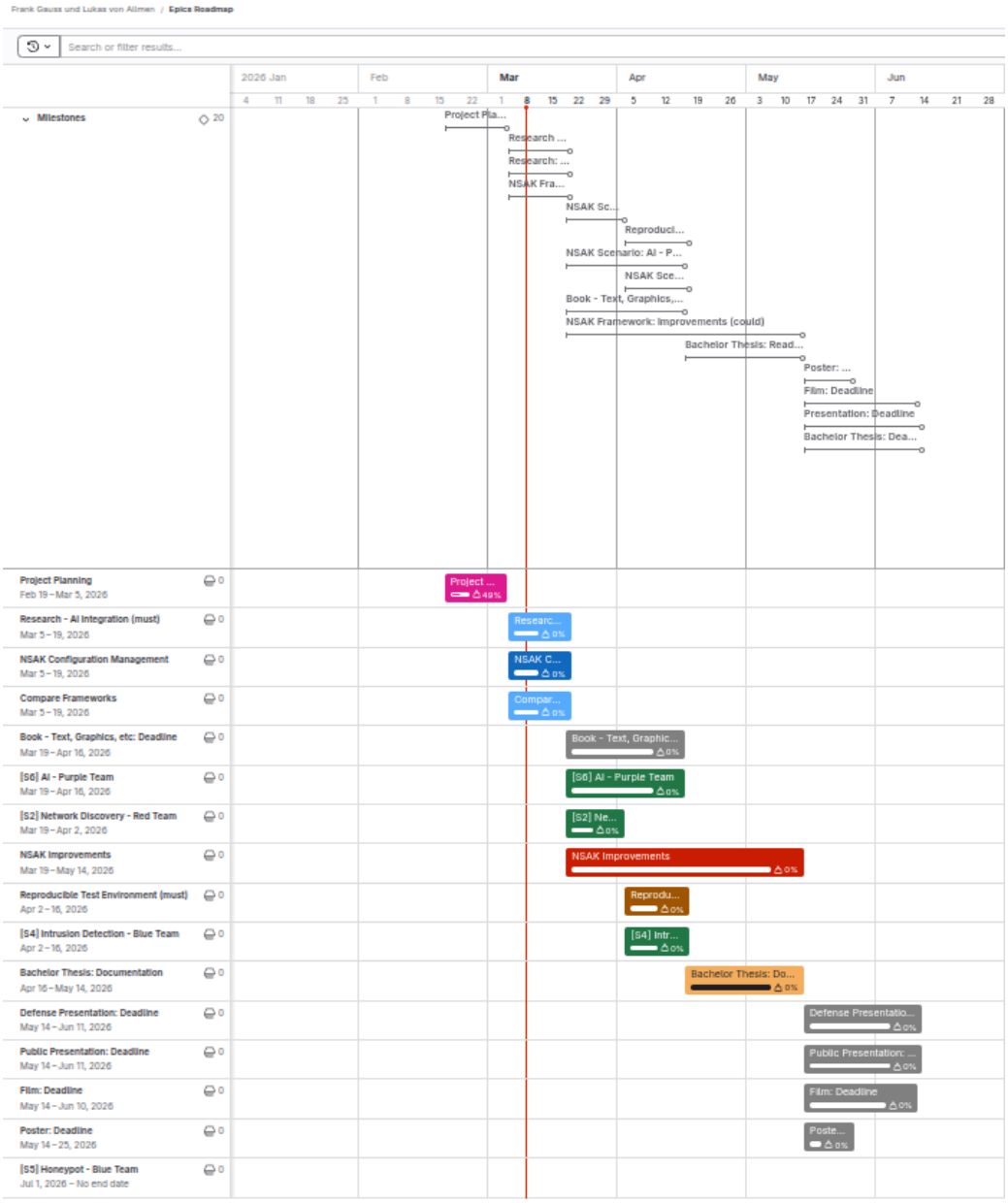

Figure A.3: Roadmap Sprint Planning 2

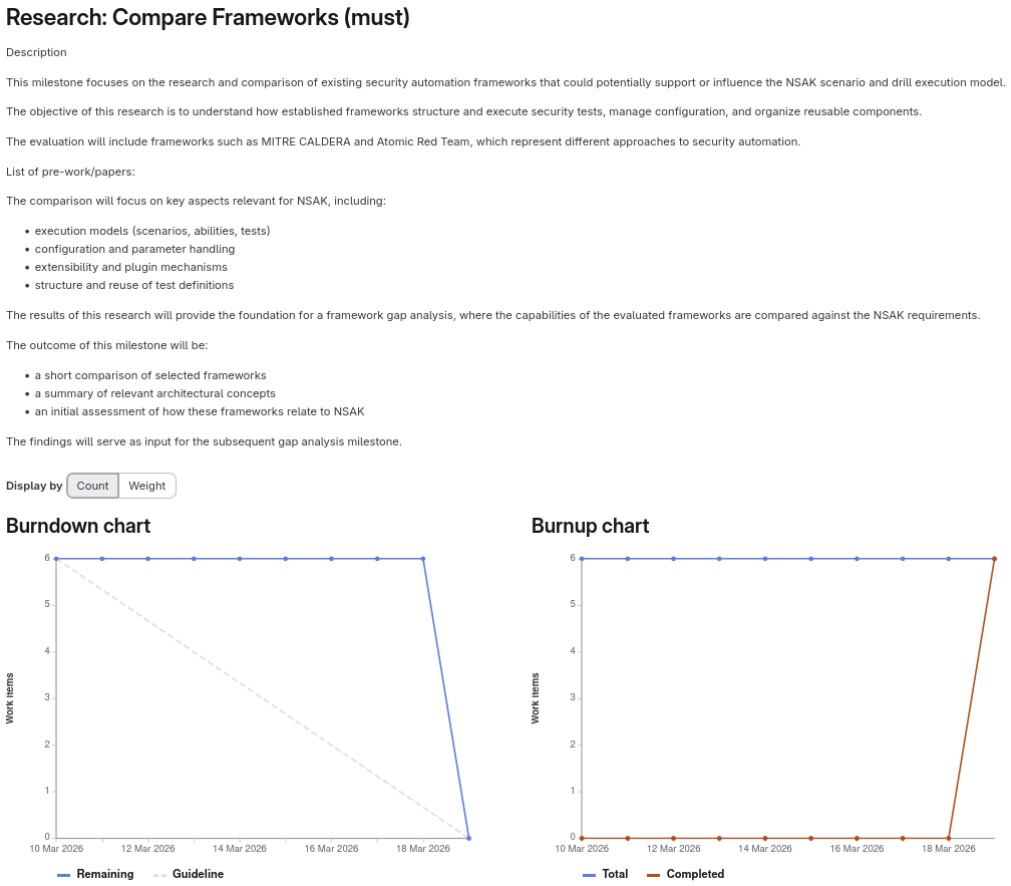

Figure A.4: Milestone Description and Charts: Framework Comparison

Add Milestone Review/Conclusion: Framework Comparsion

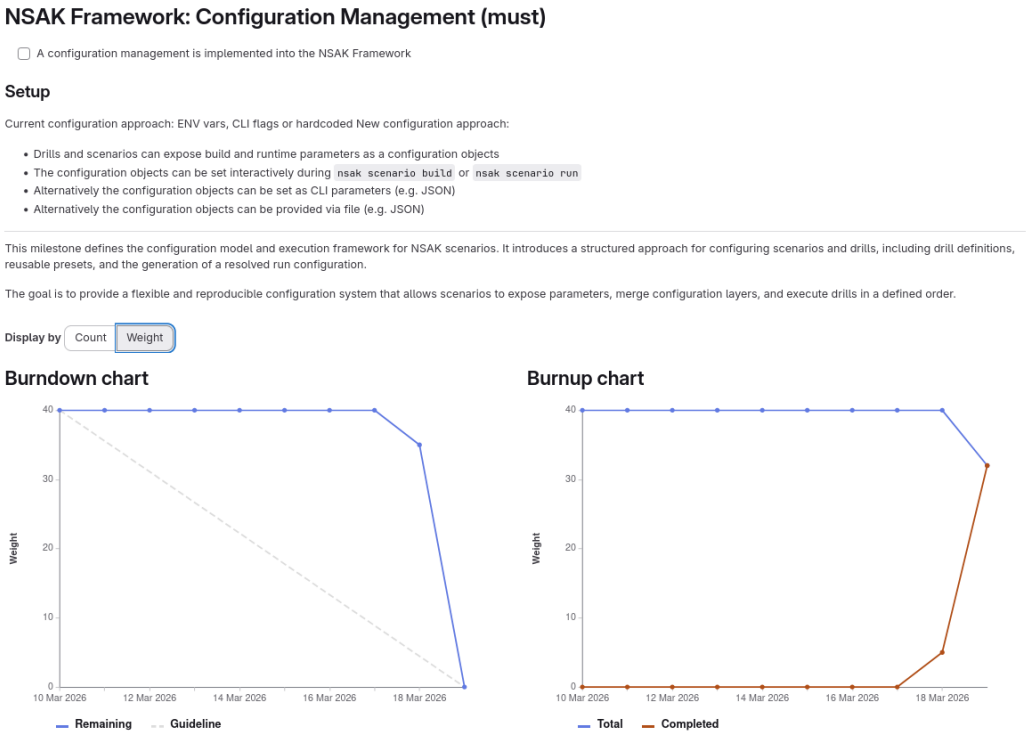

Figure A.5: Milestone Description and Charts: Configuration Management

Sprint Retro 2: 19.03.2026

Add key take-aways

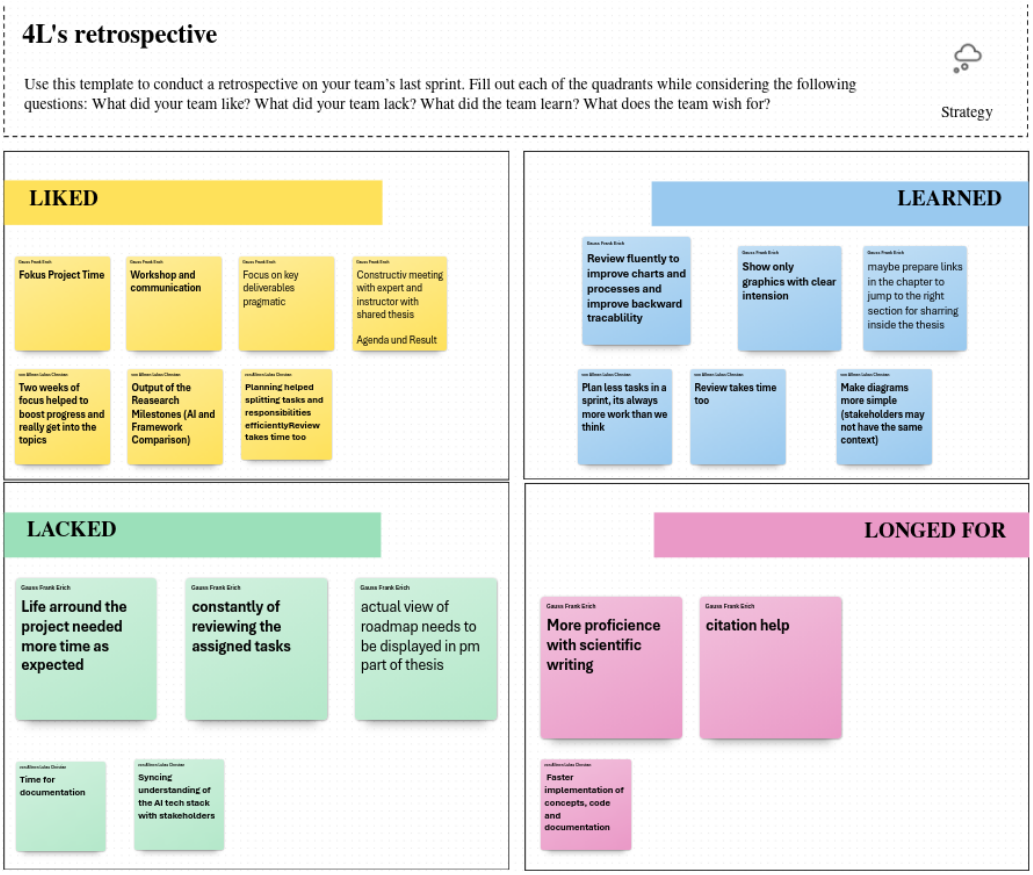

Figure A.6: Sprint Retro 2

A.7.3 Sprint 3: 19.03 –01.04.2026

Sprint Planning 3: 26.03.2026

Because we got some important input regarding variant decision for AI-integration, framework comparison, and configuration management A.16 which led to additional work. As consequence of this, we decided to shorten the third sprint to one week, which is reflected in the following table A.7 and postponed the sprint planning from the 19.03 to the 26.03. To account for the delayed start in the overall planning we adjusted t as shown in the A.7.

Week	Responsible	Task
Week 1	Frank	Reproducible Test Environment implementation
Week 1	Lukas	NSAK Scenario: AI - Purple Team (first part)

Table A.7: Sprint Planning 3

Sprint Review 3: 02.04.2026

Sprint Retro 3: 02.04.2026

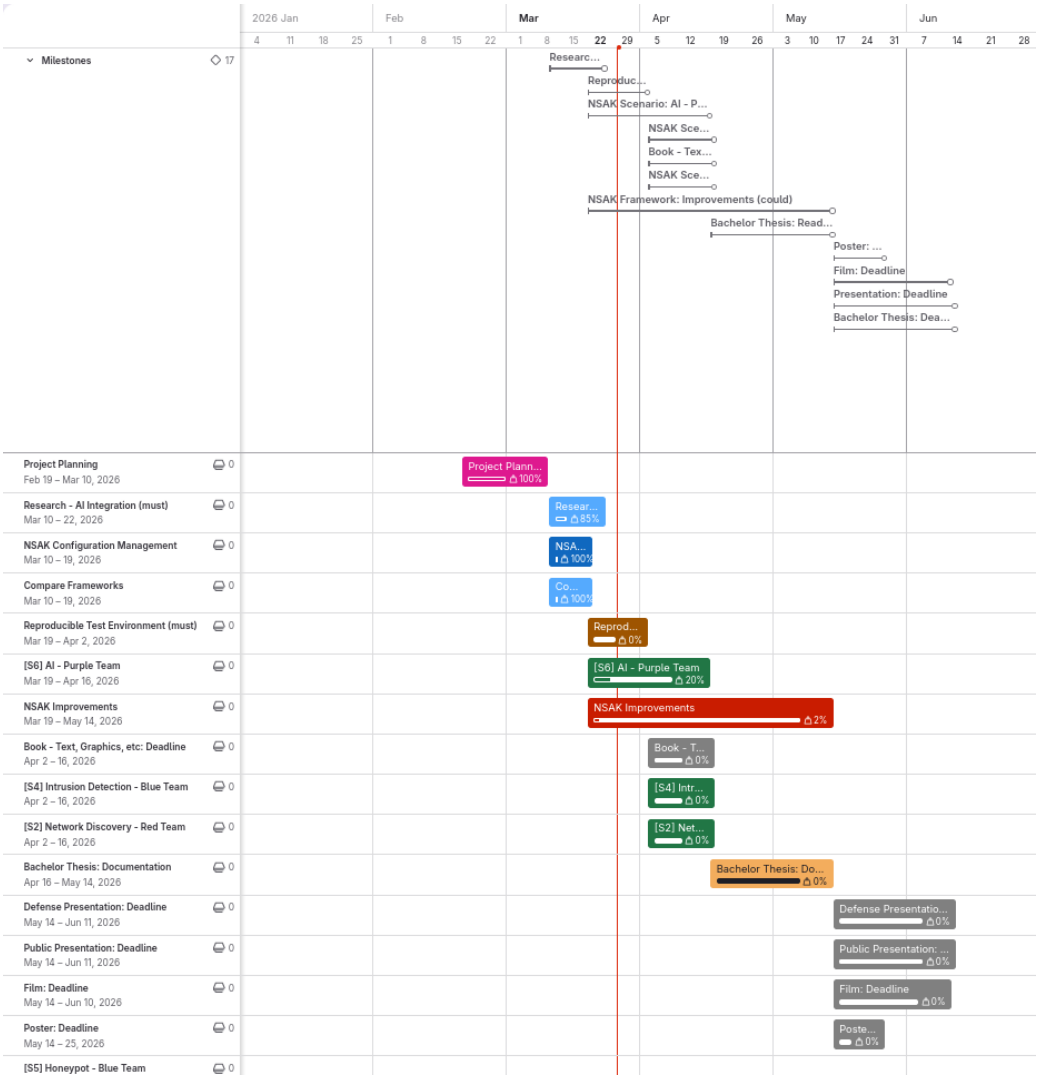

Figure A.7: Roadmap Sprint Planning 3

A.7.4 Sprint 4: 02.04 –15.04.2026

Sprint Planning 4: 02.04.2026

Sprint Review 4: 16.04.2026

Sprint Retro 4: 16.04.2026

A.7.5 Sprint 5: 16.04 –29.04.2026

Sprint Planning 5: 16.04.2026

Sprint Review 5: 30.04.2026

Sprint Retro 5: 30.04.2026

A.7.6 Sprint 6: 30.04 –13.05.2026

Sprint Planning 6: 30.04.2026

Sprint Review 6: 13.05.2026

Sprint Retro 6: 13.05.2026

A.7.7 Sprint 7: 14.05 –27.05.2026

Sprint Planning 7: 14.05.2026

Sprint Review 7: 28.05.2026

Sprint Retro 7: 28.05.2026

A.7.8 Sprint 8: 28.05 –10.06.2026

Sprint Planning 8: 28.05.2026

Sprint Review 8: 11.06.2026

Sprint Retro 8: 11.06.2026

Rewrite in past tense

A.7.9 Risk Assessment

To methodically conduct the risk assessment the section “Risk assessment” from an online guide published by Advisera [18], which describes the risk assessment process according to ISO 27001, was used as a reference.

Risk Assessment Process:

1. Risk categories and criteria: Define which dimensions and taxonomies are important for the assessment.
2. Risk identification: Collect possible risks in a brainstorming session.
3. Risk analysis: Assess the risks criteria on a risk matrix.
4. Risk evaluation: Create a list of risks ordered by the resulting risk level and evaluate if they are acceptable or need treatment.
5. Risk treatment: Describe the strategy and how to treat the risks.

Risk Categories & Criteria:

The table A.8 shows four commonly used categories, which where used to group the risks.

Category	Description
Strategic Risk	Risks related to business decisions (e.g., wrong prioritization).
Operational Risk	Risks related to internal processes (e.g., Scrum, QA process).
Financial Risk	Risks related to costs and expenses (e.g., hardware costs).
External Risk	Risks related to uncontrollable sources (e.g., force majeure).

Table A.8: Risk categories used for the project risk assessment

The criteria for analysing the risks are summarized in the table A.9

Criterion	Description
Probability	Likelihood that the risk will occur.
Impact	Severity of consequences if the risk materializes.
Level	Risk level calculated as Probability + Impact.

Table A.9: Risk evaluation criteria

The levels of the criteria and their respective numerical weights are listed in the tables A.10 and A.11.

Level	Weight	Description
Low	1	Unlikely
Medium	2	Possible
High	3	Likely

Table A.10: Risk probability scale

Level	Weight	Description
Low	1	Minor inconveniences
Medium	2	Delays milestones or reduces scope
High	3	Threat to the thesis

Table A.11: Risk impact scale

The following table A.12 defines the calculation method and the resulting risk levels, used for the assessment.

Risk Level	Weight	Calculation
Very Low	2	Low + Low
Low	3	Low + Medium
Medium	4	Medium + Medium / Low + High
High	5	Medium + High
Critical	6	High + High

Table A.12: Risk level classification (probability and impact are interchangeable)

Risk Identification

The identification process was conducted with the help of a digital brainstorming, where the risks are already grouped by category. The identified risks are represented by the cards shown in A.8. The respective risk categories are visualized by the color coding of the cards and described in the legend.

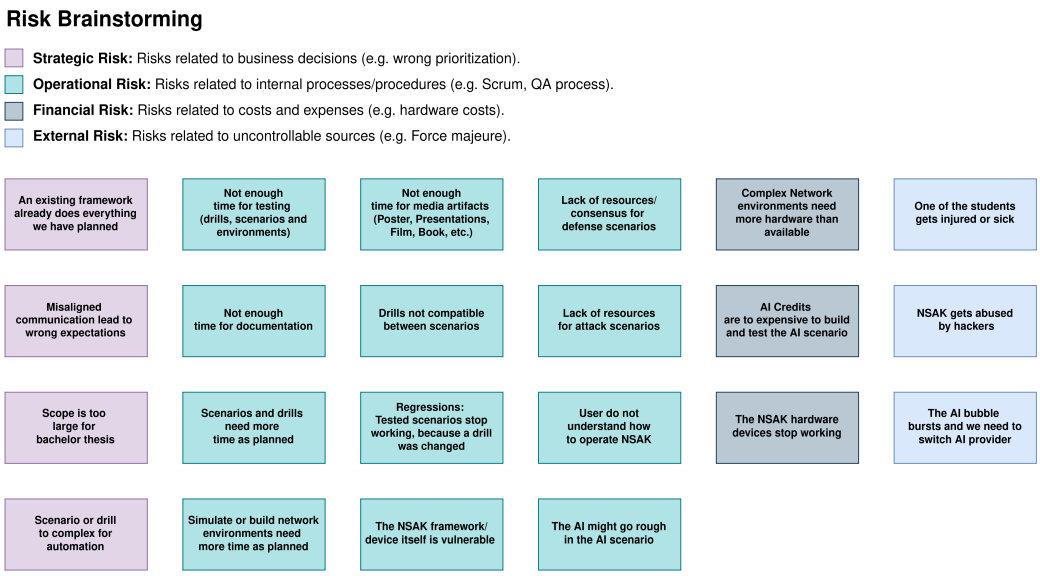

Figure A.8: Risks Brainstorming

Risk Analysis

For analysing the risks a matrix was prepared, where the criteria “Impact” and “Probability” are representing the x- and y-axis respectively. The project team then assessed the criteria for each individual risk and placed them on the matrix accordingly. In a second step, the risks where moved if they did not fit relatively to each other. The resulting “Risk Analysis Matrix” is illustrated in the following figure A.9. The background color of the cells is representing the resulting risk level of the tasks.

Risk Matrix

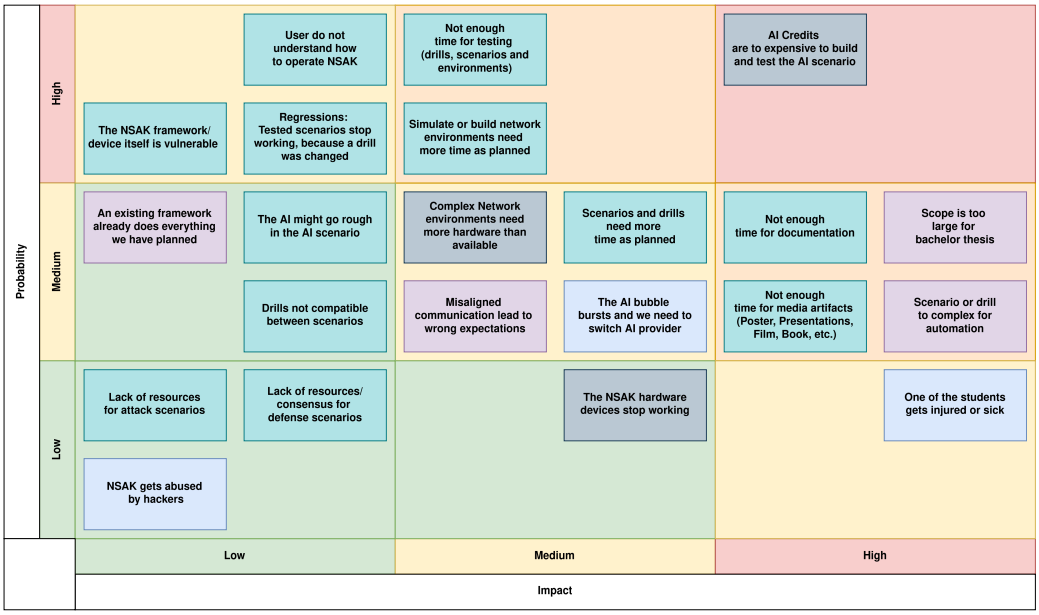

Figure A.9: Risks Brainstorming

Risk Evaluation & Treatment

In a first step a table with the required columns was created. Then the risks and weights for the criteria where added with an ID in the format “R-000”, to easier reference them in the thesis later on. After sorting the table by the calculated risk level, the risk evaluation was conducted by deciding which treatment strategy described in A.13 we want to apply.

Strategy	Description
Mitigate	Reduce the likelihood or impact of a risk.
Transfer	Move the risk to a third party.
Accept	Acknowledge the risk and take no further action.
Avoid	Eliminate or circumvent the cause of the risk.

Table A.13: Risk Treatment Strategies

Finally, the project team went through the risks and discussed how to treat them effectively. The resulting “Risk Assessment” is illustrated in the following table A.10.

Risk Assessment

Criteria: 1 = Very Low, 2 = Low, 3 = Medium, 4 = Medium, 6 = High, 9 = Critical

Level: 1 = Very Low, 2 = Low, 3, 4 = Medium, 6 = High, 9 = Critical

Category: Strategic, Operational, Financial, External

Risk ID	Description	Risk Category	Probability	Impact	Level	Strategy	Treatment
R-001	AI credits are too expensive to build and test the AI scenario	Financial	3	3	9	Avoid	Use a self-hosted LLM, check if the BRH can provide AI services or if there are free plans for education
R-006	Scope is too large for bachelor thesis	Strategic	2	3	6	Mitigate	We have to plan and evaluate the goals carefully. The use of "User Story Maps" also helps to mitigate this risk.
R-007	Scenario or drill too complex for automation	Strategic	2	3	6	Mitigate	Create "User Story Maps" for epics, so that we have a good understanding of the scenarios and their drills.
R-002	Not enough time for documentation	Operational	2	3	6	Mitigate	Add a dedicated documentation issue with an according weight to each epic.
R-003	Not enough time for testing (drills, scenarios and environments)	Operational	3	2	6	Mitigate	Add a dedicated testing issue with an according weight to each epic.
R-004	Simulating or building network environments needs more time than planned	Operational	3	2	6	Mitigate	Add dedicated issues with according weight to each scenario epic.
R-005	User does not understand how to operate NSAK	Operational	3	2	6	Mitigate	Provide a Web GUI and/or user guides
R-012	Misaligned communication leads to wrong expectations	Strategic	2	2	4	Mitigate	Multiple channels and Bi-Weekly Meetings should keep the communication aligned.
R-010	Scenarios and drills need more time than planned	Operational	2	2	4	Mitigate	Use "User Story Maps" to get a good idea of the required steps to build the scenario, it's drills and estimate generously.
R-011	Not enough time for media artifacts (poster, presentations, film, book, etc.)	Operational	2	2	4	Mitigate	Include the creation of the media artifacts as milestones/epics in the project planning.
R-009	Complex network environments need more hardware than available	Financial	2	2	4	Avoid	We can simulate network environments instead of real hardware.
R-008	The AI bubble bursts and we need to switch AI provider	External	2	2	4	Avoid	Use vendor agnostic tools and protocolls, so that we can switch provider easily.
R-014	Regressions: tested scenarios stop working because a drill was changed	Operational	3	1	3	Mitigate	Add automated test cases for detecting regressions early.
R-015	The NSAK framework/device itself is vulnerable	Operational	3	1	3	Transfer	Clearly state that the NSAK framework/device is still a POC and not ready for production use.
R-013	One of the students gets injured or sick	External	1	3	3	Accept	We accept this risk.
R-019	An existing framework already does everything we have planned	Strategic	2	1	2	Accept	We accept the risk, but will compare well known frameworks with NSAK and highlight our USP (HW and AI).
R-017	Drills not compatible between scenarios	Operational	2	1	2	Mitigate	The implementation of the configuration management should provide a way that small deviations can be branched.
R-018	The AI might go rogue in the AI scenario	Operational	2	1	2	Mitigate	Besides the fact that we have physical access to the device we add a killswitch to the AI scenario.
R-016	The NSAK hardware devices stop working	Financial	1	2	2	Accept	The fact that we have two devices already reduces the impact, further we could also use other HW.
R-021	Lack of resources for attack scenarios	Operational	1	1	1	Mitigate	Get familiar and use the MITRE ATT&CK framework as reference and source.
R-022	Lack of resources / consensus for defense scenarios	Operational	1	1	1	Mitigate	Get familiar and use well known sources such as NIST as source.
R-020	NSAK gets abused by hackers	External	1	1	1	Transfer	Clearly state that the NSAK framework/device is built only for educational and white hate purposes.

Figure A.10: Risk Assessment

A.8 Meeting Notes

A.8.1 Checkpoint Meeting 1: 26.02.26

Add slides if we can reproduce them

Participants	Hj. Wenger (Instructor), Frank Gauss, Lukas von Allmen
Agenda	- Goals presentation via slides - Clarification whether there is a deadline or submission required for the presentation (besides adding it to the addendum). - Clarification whether the defense presentation should be an enhanced version of the public presentation or a separate, more technical presentation. - Proposal to establish a recurring meeting series every Thursday at 15:00 during the semester.
Tasks	- Hand in a set of success criteria for the project goals: We thought we can delivery the success criteria and the project goals as milestones in gitlab, we corrected this later on. - Conduct a risk assessment: A.7.9 - Contact expert via e-mail: Contacted at 21.02.26, response 03.03.26, invited to next checkpoint meeting 05.03.26, the expert takes part at the meeting every two weeks

Table A.14: Meeting Notes: Checkpoint 1

A.8.2 Checkpoint Meeting 2: 05.03.26

Add slides if we can reproduce them

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Review of the project goals as milestones and project planning. - Review of risk assessment. - Discussion on the investigation and analysis of network communication between end devices and external networks (RED scenario: sniffing, filtering, and potential data exfiltration).
Decisions	- Variant 1 was selected for further development. - Open questions remain regarding the use and management of AI tokens.
Notes	- We misunderstood how to deliver the project goals and success criteria: Instead of defining the goals as milestone we added a goal section in the thesis. - Discussion on the rationale behind selecting Variant 1. Consideration of whether a user interface is required when MCP functionality is available - Open Research Question. - NSAK integrates AI capabilities. - The development of a Web GUI is currently considered lower priority - suggestion of Urs Keller.
Tasks	- Send the slides of the final presentation and documentation of “Project 2” to Urs Keller (Expert). - Delivery the projects goals with a detailed description as part of the thesis by Saturday, 07.03.26: 1.3 - Finalize the project planning: A.6 - Write the first few chapters of the thesis:

Table A.15: Meeting Notes: Checkpoint 2

A.8.3 Checkpoint Meeting 3: 19.03.26

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Review Project Management and Roadmap - Review of the active milestones (Framework Comparison, AI-Research, Configuration Management) - Discuss Variant Decisions: AI-Integration - Ask about AI token usage - Ask about repository access
Decisions	- We will proceed using NSAK as the basis for our thesis. - Open questions remain regarding the use and management of AI tokens.
Notes	- -
Tasks	- Document decision that we will continue to work with the NSAK framework - Expand on variant decision for AI-Integration: - Split diagram into 2 variants - Expand in detail how the variants are working (e.g., sequence diagram) - Configuration management: Move metadata and others below the device node - BFH TI: Has a cluster with studio macs for AI usage -> Ask if we can use them (https://mlmp.ti.bfh.ch/ via VPN)

Table A.16: Meeting Notes: Checkpoint 3

A.8.4 Checkpoint Meeting 4: 02.04.26

A.8.5 Checkpoint Meeting 5: 16.04.26

A.8.6 Checkpoint Meeting 6: 30.04.26

A.8.7 Checkpoint Meeting 7: 14.05.26

A.8.8 Checkpoint Meeting 8: 28.05.26

A.8.9 Checkpoint Meeting 9: 11.06.26

Variant 1: AI Purple Team

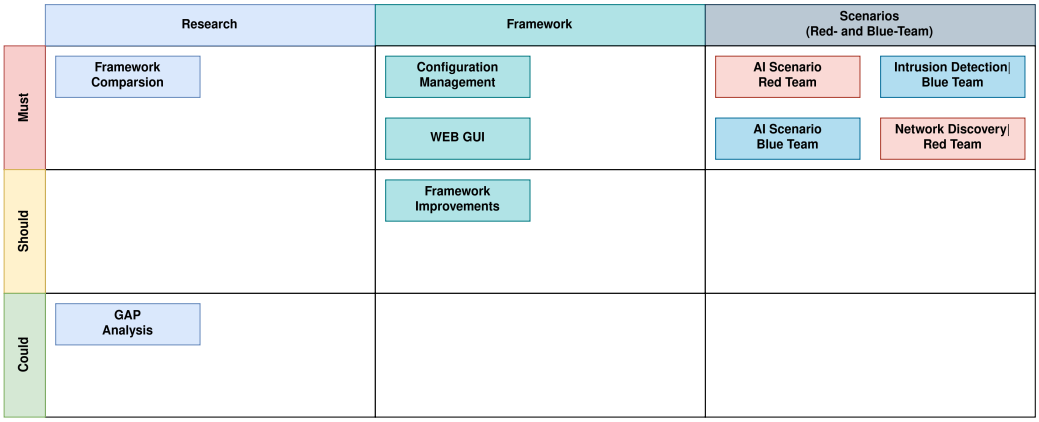

Figure A.11: Variant 1: AI vs Manually Built Scenarios

A.9 Variant Decision: Thesis Goals

During the first sprint of the project, which was focused on project management and planning, we identified the very promising hypothesis that red and blue team activities could be automated and/or enhanced by AI. Pursuing this hypothesis would change the projects focus and goals and thus requires the agreement and approval of the stakeholder. To communicate this circumstance efficiently and leave the option to go with the status quo, we prepared two options to conduct a "Variant Decision". In the following two subsections the two variants presented in the second checkpoint meeting A.15 are described and illustrated. It must be noted, that the variants and their visualizations A.11 A.12 are momentary snapshots and will not be updated for traceability reasons. For example, the Web GUI was already deprioritized during the discussion of the variants, which is reflected in the project goals 1.3 but not here.

A.9.1 Variant 1: AI vs Manually Built Scenarios (Pivot)

Figure A.11 shows the project goals adjusted for the AI focused hypothesis, which is favored variant of the project team. In this option most of the evaluated scenarios will not be realized in favor of AI based scenarios, which also requires additional research 1.3.1. The basic idea is to implement scenarios for network discovery 1.3.3, intrusion detection 1.3.3 and an AI based scenario with the same capabilities 1.3.3. In the evaluation of the thesis we can then assess the quality of the AI based scenarios itself 1.3.4 and in comparison to the manually built scenarios 1.3.4.

Variant 2

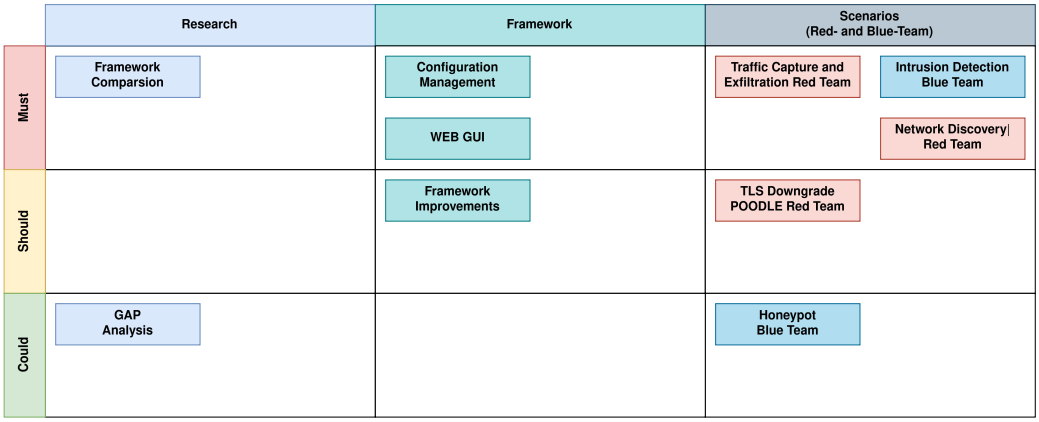

Figure A.12: Variant 2: Multiple Blue and Red Team Scenarios

A.9.2 Variant 2: Multiple Blue and Red Team Scenarios (Status Quo)

This is the fallback if the other option A.9.1 is not approved by the stakeholders. Even if the project team does not favor this more conservative variant, it still represents a valid way to conduct an interesting project. As illustrated below A.12, in this variant we propose to proceed with the previously planned scenarios for which we already conducted "User Story Mappings" documented under the workshops section**??**. The focus would lie in more conventional scenarios, overall framework improvements and an in depth comparison with another framework in form of a GAP Analysis.

A.9.3 Decision

The stakeholders and project team decided in favor of **Variant 1: AI vs Manually Built Scenarios (Pivot)** A.9.1 in the checkpoint meeting A.15.